

Crabs on camera: code documentation

Gina Brøten

2026-08-05

Background

Project and repository aims

The basis of this project is rooted in the work done in the masters thesis by Brøten (2026). The aim of the project was to improve monitoring of the data-limited brown crab (*Cancer pagurus*) stock in Norway using machine learning and underwater video surveys.

The aim of this repository is not to push the boundaries of machine learning or showcase new methods, rather to make these methods more approachable for marine scientists working in fisheries applications.

About the repository

This repository uses the same code, functions, and packages developed for the masters thesis, though some functions have been simplified here to make the pipeline easier to reproduce.

What does this repository contain:

To try it yourself, clone this repository, either on your own computer for the small example dataset, or on a server with GPU access if you plan to train on a larger dataset.

AI tools (large language models) were used throughout this repository to help generate code, fix syntax, and clean up errors.

Glossary

Object detection Bounding box Mask Annotation Label Class Stratified sampling YOLO

Before running the scripts

Prerequisites

All packages required for data transformation and object detection training are open-source and freely available.

To identify and quantify crabs in the videos, this project uses the object detection framework “You Only Look Once” (YOLO), specifically version 8 (Jocher, Qiu, and Chaurasia 2023), which builds on the original YOLO architecture introduced by Redmon et al. (2016). Several object detection frameworks, including newer YOLO versions, could have been used instead. YOLOv8 was chosen because it is well documented and its common issues are well known, which makes it easier to troubleshoot for beginners in machine learning.

Python must be installed. `raw_to_yolo.ipynb` checks for missing packages and installs them automatically (see Chunk 0).

A server with GPU access is recommended if you plan to train on a large dataset — the small example dataset shipped with this repo can be run locally without one.

If you want to train a detector on your own data, you'll need an annotated dataset structured the same way as the example dataset: an `annotations` folder containing the CVAT/COCO export (`instances_default.json`), and an `images/default` folder containing the matching image files.

Data

The repo only ships a small example/test dataset (`data/annotations/test_annotations`) so the scripts can be run and verified without downloading everything.

The dataset used here for testing is from the Institute of Marine Research inshore gill- and fyke-net survey conducted in 2025. The material is from five Baited Remote Underwater Video (BRUV) stations over 541 images. For more stats about the annotation composition, run `annotation_stats`. Annotations are exported from CVAT, as a `.JSON` file, which is a plain text file storing structured data, in this case annotations, and such as categories (here these are species groups).

The images are labelled with multiple identifiers to make analysis and stratified sampling later simpler.

Ex. `NO_2025_3007_GarnRuse_Lovund_LO3_RA1_L9_frame_9`, is collected in Norway, in 2025 on the 30.07 on the Garn and Ruse Survey, the location name is Lovund, and is station number is LO3, and the BRUV-rig used is L9, and this is video number 9 from the left camera (L9), and is frame number 9.

So if you're applying another type of video-survey or other naming logic please be mindful, though it's possible to turn off this stratified sampling in the functions. If you are annotating your own dataset, a JSON file for the structure applied in this annotation process can be found in (`script/annotation_format`).

For more in depth information about the sampling methods and areas, annotation choices and a discussion of methods can be found in Brøten (2026). # From raw annotations to YOLO-ready data

Important mention, if you are applying another dataset than the test-dataset please be mindful of but not limited to - File paths - Stratified sampling for k-folds

The goal of this script is to go from a JSON file and images, to a five fold stratified data set where the RLE-masks are converted to bounding boxes, where each image has its corresponding `.txt` document containing the (possible) annotation(s) and is ready for object detection training. The converted data set is split into folds, for maximizing the use of a limited dataset, the data is split following a random stratified sampling process.

1. Chunk 0 – 3

The first chunks download and loads the necessary packages, and then finds and loads the correct working directory containing the annotated images and the annotations. The code chunks should run without any error (utdtype?)

If you are using another data set `CONFIG` in Chunk 2 must be changed to the correct directory and file paths.

2. Chunk 4

Training object detection models with RLE-masks requires more run-time, and is ideal for object segmentation purposes. For a project like this where the sole goal is to identify (and quantify) crab in the images opting for the simplest version is preferable.

A bounding box is a rectangular boundary defined by four coordinates, whereas RLE-mask is a compressed binary pixel outlining the exact shape of an object. For object detection, bounding boxes are preferred, and to reduce run time since they are more lightweight.

RLE annotation is faster as it's powered by Segment Anything Model in CVAT, making the already tedious annotation process a bit smoother.

In this chunk, the RLE-masks get converted into tightly wrapped bounding boxes, and the coordinates for the bounding boxes are normalized to between the expected values from YOLO (0 – 1)

All other species annotated that's not Decapod & crab, keeps only one. Duplicated bounding boxes also get removed.

3. Chunk 5

Links the images to the annotations, if you are using another dataset it is important that the names of the images and their corresponding .txt document with the annotations have the same name. Unmatched output means that there might be a typo, renamed files or missing files in the folders.

NB, training on negative examples, empty frames has shown to improve performance, so they are kept in the data set with a empty corresponding .txt document.

4. Chunk 6 – 7

These chunks serve as a quality and safety checks, showing the first annotated image in the dataset

Here, all the crabs in the image should have a red box tightly wrapping the crab

Likewise for the RLE, the

This function follows the naming logic to split up into locations and stations

Chunk 8

Location and BRUV-station helpers

Since the data is station-based we need to be careful of data leakage, where if images from the same stations appear in both training and in validation, the model does not actually learn the features but rather just memorizes them.

Chunk 9 – 10

Traditional machine learning models split the dataset into two parts, one part (usually 80%) is reserved for training, and the remaining (20%) is for validating the training, but this means this remaining part is never used for training. Most approaches to machine learning models are moving away from this approach and towards k-fold cross validation, where the data is split into k non-overlapping subsets, where each fold consists of one subset for validation and the remaining subsets are reserved for training. In this instance there are five folds, but this can easily be changed in the function.

Since the data set is imbalanced, has uneven representation of crabs and is station-based, stratified sampling is applied for the sampling of the folds and subunits.

Since the test dataset is not large, perfectly balanced folds is impossible.

If everything looks ok, then the folds are ready for training.

References

- Brøtten G. Crabs on camera: Improving the monitoring of the brown crab (*Cancer pagurus*) with machine learning and Baited Remote Underwater Video (BRUV). Master's thesis, University of Bergen, 2026. <https://hdl.handle.net/11250/5537346>.
- Jocher G, Qiu J, Chaurasia A. *Ultralytics YOLO*. 2023. <https://github.com/ultralytics/ultralytics>.
- Redmon J, Divvala S, Girshick R *et al.* *You Only Look Once: Unified, Real-Time Object Detection*. arXiv, 2016. <https://doi.org/10.48550/arXiv.1506.02640>.
- Sekachev B, Manovich N, Zhiltsov M *et al.* *Opencv/Cvat: V1.1.0*. 2020. <https://zenodo.org/records/4009388> (7 Aug. 2026, date last accessed).